Tecnología WiFi con Raspberry Pi Pico 2W.

Paula Quintero y Ediem Valero.
Comunicación Digital.
Universidad Militar Nueva Granada
Bogotá, Colombia
Noviembre 04, 2026.

Resumen – En esta práctica de laboratorio se implementaron y analizaron diferentes funcionalidades de comunicación inalámbrica utilizando tarjetas Raspberry Pi Pico 2W y módulos NRF24L01. Inicialmente, se exploraron las capacidades WiFi de la Raspberry Pi mediante la identificación de su dirección MAC, el escaneo de redes disponibles y la implementación de un servidor web para controlar un LED desde una interfaz HTML. También se realizaron pruebas de lectura del convertidor analógico-digital (ADC), medición de la intensidad de señal recibida (RSSI) a diferentes distancias y observación de la señal inalámbrica mediante un analizador de espectro.

Luego, se integraron dos módulos NRF24L01 con sus respectivas Raspberry Pi, estableciendo una comunicación de transmisión y recepción de datos. Se desarrolló una interfaz web que permite configurar parámetros de radiofrecuencia como el canal, la velocidad de transmisión, la potencia, el modo de operación y el mensaje que se desea enviar. Esta configuración se refleja también en las pantallas OLED de los dispositivos, conservando su función de visualización. Finalmente, se comprobó la recepción de mensajes entre ambos módulos, incluso cuando las Raspberry Pi estaban conectadas a redes WiFi diferentes, evidenciando que la comunicación mediante NRF24L01 funciona de manera independiente de la conexión WiFi utilizada para acceder a la interfaz de configuración.

Palabras clave: Raspberry Pi Pico W, comunicación inalámbrica, WiFi, NRF24L01, RSSI, transmisión y recepción de datos.

I. INTRODUCCIÓN

Actualmente, las comunicaciones inalámbricas hacen parte de muchos de los dispositivos y sistemas que utilizamos diariamente, ya que permiten intercambiar información sin necesidad de utilizar cables. Tecnologías como WiFi y los módulos de radiofrecuencia facilitan la conexión entre equipos y permiten desarrollar aplicaciones de control y transmisión de datos. Por esta razón, es importante conocer cómo funcionan estas tecnologías y cómo se pueden implementar utilizando dispositivos programables.

En este laboratorio se trabajó con tarjetas Raspberry Pi Pico 2W y módulos NRF24L01, con el propósito de explorar diferentes formas de comunicación inalámbrica y poner en práctica los conocimientos adquiridos en el área de telecomunicaciones. Inicialmente, se realizaron pruebas con la conexión WiFi de la Raspberry Pi, como la identificación de redes disponibles, la medición de la intensidad de señal (RSSI) a diferentes distancias y la observación de la señal mediante un analizador de espectro. También se desarrolló una página web que permitió controlar el LED de la tarjeta y realizar algunas pruebas de funcionamiento.

En la segunda parte del laboratorio se trabajó con dos módulos NRF24L01 conectados a sus respectivas Raspberry Pi, con el objetivo de establecer una comunicación entre ellos mediante el envío y la recepción de mensajes. Durante esta etapa se configuraron diferentes parámetros, como el canal de operación, la velocidad de transmisión, la potencia y el tipo de envío. Además, se integró una página web para modificar estas opciones desde el navegador, reemplazando el uso de los botones físicos para la configuración, mientras que las pantallas OLED se conservaron para visualizar los parámetros seleccionados.

Uno de los aspectos más importantes de esta práctica fue comprobar que los módulos NRF24L01 podían comunicarse entre sí, aunque las Raspberry Pi estuvieran conectadas a redes WiFi diferentes, debido a que utilizan un enlace de radiofrecuencia independiente de la conexión WiFi. De esta manera, se pudo observar cómo se pueden combinar distintas tecnologías inalámbricas para desarrollar un sistema de comunicación y control.

Finalmente, en este informe se presentan los procedimientos realizados durante las dos partes del laboratorio, las configuraciones utilizadas, las pruebas desarrolladas y los resultados obtenidos. También se incluye el análisis del comportamiento de las comunicaciones implementadas, con el fin de identificar su funcionamiento y comprender mejor las características de cada tecnología.

II. OBJETIVOS

Objetivo general: Implementar y analizar sistemas de comunicación inalámbrica mediante tarjetas Raspberry Pi Pico 2W y módulos NRF24L01, realizando pruebas de conectividad WiFi, medición de parámetros de señal y transmisión y recepción de datos, con el fin de comprender el funcionamiento y las características de estas tecnologías.

Objetivos específicos:

- Explorar las funciones de conectividad WiFi de la Raspberry Pi Pico 2W mediante la identificación de redes

disponibles, la lectura de la dirección MAC y la implementación de un servidor web.

- Medir la intensidad de la señal recibida (RSSI) a diferentes distancias para observar cómo varía la calidad de la señal inalámbrica con respecto a la separación entre los dispositivos.
- Observar y analizar las características de una señal WiFi mediante el uso de un analizador de espectro, identificando parámetros como la frecuencia y el nivel de potencia.
- Implementar un sistema de comunicación inalámbrica entre dos tarjetas Raspberry Pi mediante módulos NRF24L01, configurando parámetros como el canal, la velocidad de transmisión y la potencia.
- Desarrollar una interfaz web que permita modificar los parámetros de configuración de los módulos NRF24L01 y controlar el envío de mensajes desde un navegador.
- Comprobar la transmisión y recepción de mensajes entre los dos módulos NRF24L01, incluso cuando las Raspberry Pi se encuentran conectadas a redes WiFi diferentes.

III. MARCO TEÓRICO

1. Comunicaciones inalámbricas:

Las comunicaciones inalámbricas permiten transmitir información entre dispositivos sin necesidad de utilizar conexiones físicas, mediante ondas electromagnéticas que se propagan a través del espacio. Estas tecnologías se utilizan en diferentes aplicaciones, como redes de computadores, sistemas de control remoto, sensores y dispositivos de Internet de las Cosas (IoT). Entre las tecnologías más utilizadas se encuentran WiFi y los sistemas de comunicación por radiofrecuencia, los cuales permiten intercambiar información utilizando diferentes protocolos y configuraciones. [1]

2. Raspberry Pi Pico 2W:

La Raspberry Pi Pico W es una tarjeta de desarrollo basada en el microcontrolador RP2040, que incorpora conectividad inalámbrica WiFi en la banda de 2,4 GHz. Permite desarrollar aplicaciones electrónicas y de comunicación mediante lenguajes de programación como MicroPython, facilitando el control de periféricos, la lectura de sensores y la conexión con redes inalámbricas.

Una de sus principales características es que puede funcionar como cliente WiFi, conectándose a una red existente, o como punto de acceso (Access Point), creando una red propia a la que pueden conectarse otros dispositivos. Esto permite implementar aplicaciones de control y comunicación sin necesidad de utilizar hardware adicional para gestionar la conexión inalámbrica. [2]

3. Dirección MAC y escaneo de redes Wifi:

La dirección MAC (Media Access Control) es un identificador asociado a la interfaz de red de un dispositivo, utilizado para identificarlo dentro de una red local. Generalmente está representada por seis grupos de dos caracteres hexadecimales.

Por otra parte, el escaneo de redes WiFi permite identificar los puntos de acceso disponibles dentro del alcance de un dispositivo. Durante este proceso se pueden obtener datos como el nombre de la red (SSID), el canal de operación, la dirección MAC del punto de acceso y la intensidad de la señal recibida.

Esta función permite conocer las redes inalámbricas cercanas y observar las condiciones del entorno en el que se realizan las pruebas. [3]

4. Servidor web y protocolo HTTP:

Un servidor web es un programa que recibe solicitudes de otros dispositivos y responde enviando información o ejecutando determinadas acciones. En el laboratorio se implementó un servidor web directamente en la Raspberry Pi Pico W, utilizando MicroPython y el módulo de comunicación por sockets.

Para el intercambio de información se utilizó HTTP (Hypertext Transfer Protocol), un protocolo que permite la comunicación entre clientes y servidores web. Mediante solicitudes HTTP, un navegador puede solicitar una página o enviar instrucciones al servidor.

En la práctica, el servidor permitió alojar una página web desde la cual se podía controlar el LED de la Raspberry Pi y, posteriormente, configurar los parámetros de comunicación del módulo NRF24L01.

5. Convertidor analógico-digital (ADC):

El convertidor analógico-digital (ADC) es un componente que permite transformar una señal eléctrica analógica en un valor digital que puede ser procesado por un microcontrolador. Este proceso consiste en tomar una muestra de la señal y representarla mediante un valor numérico, de acuerdo con la resolución del convertidor y su voltaje de referencia.

La Raspberry Pi Pico W dispone de entradas analógicas que permiten realizar lecturas de voltaje mediante el ADC. En el laboratorio se utilizó esta característica para obtener valores de una entrada analógica y visualizar las mediciones a través de la interfaz web. [4]

6. Intensidad de señal recibida (RSSI):

El RSSI (Received Signal Strength Indicator) es un indicador que representa la intensidad de una señal inalámbrica recibida por un dispositivo. Generalmente se expresa en dBm y sus valores suelen ser negativos. Cuanto más cercano a cero sea el valor, mayor será la potencia recibida.

La intensidad de señal puede variar dependiendo de factores como la distancia entre los dispositivos, los obstáculos presentes, las interferencias y las condiciones del entorno. Por esta razón, el RSSI es un parámetro útil para evaluar el comportamiento de una conexión inalámbrica y observar cómo cambia la señal al modificar la separación entre el transmisor y el receptor. [5]

7. Módulo de comunicación NRF24L01:

El NRF24L01 es un transceptor inalámbrico que permite transmitir y recibir datos mediante radiofrecuencia en la banda ISM de 2,4 GHz. Es un dispositivo de bajo consumo y tamaño reducido, utilizado en proyectos de comunicación entre microcontroladores y sistemas embebidos.

Este módulo permite configurar diferentes parámetros de funcionamiento, como el canal de frecuencia, la velocidad de

transmisión y la potencia de salida. También permite trabajar en dos modos principales: transmisión (TX) y recepción (RX), lo que facilita el intercambio de información entre dos dispositivos.
En el laboratorio se utilizaron dos módulos NRF24L01 conectados a sus respectivas tarjetas Raspberry Pi, con el propósito de establecer un enlace inalámbrico para el envío y recepción de mensajes. [6]

8. Integración de WiFi y NRF24L01:
La integración de WiFi con los módulos NRF24L01 permite combinar dos tecnologías inalámbricas que cumplen funciones diferentes. WiFi se utilizó para establecer la conexión entre la Raspberry Pi y el dispositivo desde el cual se accede a la página web, mientras que el NRF24L01 se encargó de realizar la transmisión y recepción de mensajes entre las dos tarjetas.
De esta manera, cada Raspberry Pi puede conectarse a una red WiFi diferente y mantener la comunicación entre sus módulos NRF24L01 mediante un enlace de radiofrecuencia independiente. La interfaz web facilita la configuración de los parámetros de transmisión y recepción, mientras que las pantallas OLED permiten visualizar la información de cada dispositivo.

IV. PROCEDIMIENTO

1. Primera parte: Implementación y pruebas de conectividad WiFi con Raspberry Pi Pico W

1.1. Identificación de la dirección MAC:
Inicialmente, se conectó la Raspberry Pi Pico W al computador y se utilizó el entorno de programación Thonny para trabajar con MicroPython. Se desarrolló un código para acceder a la interfaz inalámbrica de la tarjeta y consultar su dirección MAC mediante las funciones disponibles en la biblioteca network.
Luego, se ejecutó el programa y se consultó la dirección MAC desde la consola de Thonny. También se realizaron reinicios de la tarjeta para verificar que el procedimiento de lectura pudiera repetirse.

1.2. Escaneo de redes WiFi disponibles:
Se implementó un programa en MicroPython para realizar un escaneo de las redes inalámbricas cercanas utilizando la interfaz WiFi de la Raspberry Pi Pico W.
Para esto, se activó la interfaz inalámbrica y se utilizó la función de escaneo de redes, con el propósito de identificar los puntos de acceso disponibles. La información obtenida se organizó para consultar características como el nombre de la red (SSID), la dirección MAC del punto de acceso, el canal de operación y la intensidad de señal recibida.
El procedimiento se ejecutó varias veces para observar las redes detectadas durante cada exploración.

1.3. Implementación de un punto de acceso y servidor web:
Se configuró la Raspberry Pi Pico 2W para funcionar como punto de acceso WiFi.. Para ello, se establecieron los parámetros correspondientes al nombre de la red, la contraseña, el canal de operación y la dirección IP local.
Una vez activada la red inalámbrica, se desarrolló un servidor web utilizando MicroPython y las bibliotecas network y socket. El servidor se configuró para recibir solicitudes HTTP desde un navegador y entregar una página web almacenada en la memoria de la Raspberry Pi.
Se abrió el archivo index.html, que tiene una interfaz con dos botones que permitieron encender y apagar el LED integrado de la tarjeta. Mediante solicitudes HTTP, la página enviaba las instrucciones al servidor, el cual interpretaba cada petición y modificaba el estado del LED.
Finalmente, se conectó un dispositivo a la red creada por la Raspberry Pi y se ingresó a la dirección IP del servidor desde el navegador para utilizar los controles de la página web.

1.4. Observación de la señal WiFi mediante el analizador de espectro:
Para realizar la observación de la señal inalámbrica se utilizó un analizador de espectro configurado para trabajar en la banda de 2,4 GHz.
Se estableció un intervalo de observación (Span) de 40 MHz, comprendido entre 2412 MHz y 2452 MHz. Luego, se activó la conexión WiFi de la Raspberry Pi y se realizaron observaciones de la actividad de radiofrecuencia utilizando la función de barrido del instrumento.
También se utilizó el marcador para seleccionar componentes de la señal y consultar sus parámetros de frecuencia y nivel de potencia. Durante este procedimiento se realizaron observaciones con el propósito de identificar la actividad de transmisión inalámbrica dentro del intervalo configurado.

1.5. Lectura del convertidor analógico-digital (ADC):
Se realizó una prueba de lectura analógica utilizando la entrada ADC0 de la Raspberry Pi Pico W, correspondiente al pin GP26. Para ello, se programó la tarjeta mediante MicroPython, utilizando la biblioteca machine para inicializar el convertidor analógico-digital y obtener las lecturas de la entrada. Luego, se incorporó esta lectura al servidor web para permitir su consulta desde la página HTML.
De esta manera, se integró la lectura de una variable analógica con la interfaz web desarrollada anteriormente.

1.6. Medición de la intensidad de señal recibida (RSSI) en función de la distancia:
Para estudiar el comportamiento de la señal WiFi con respecto a la distancia, se realizaron mediciones utilizando la Raspberry Pi Pico 2W y una red inalámbrica disponible.
Se establecieron diferentes distancias entre el dispositivo y el punto de acceso, tomando mediciones de intensidad de señal en cada posición. Para organizar la información, se realizaron varias lecturas por distancia y se almacenaron los datos en archivos CSV.
Luego, se desarrollaron mediciones adicionales para explorar el alcance máximo de la conexión inalámbrica. Para esta prueba se desplazó el dispositivo progresivamente hasta la distancia de interés y se registraron las lecturas correspondientes.
Finalmente, los archivos CSV se utilizaron para organizar los datos y elaborar gráficas en Microsoft Excel, relacionando la distancia con la intensidad de señal recibida.

2. Segunda parte: Implementación de pruebas de comunicación con módulos NRF24L01

2.1. Conexión de los módulos NRF24L01 con las Raspberry Pi:

Para esta parte del laboratorio se utilizaron dos tarjetas Raspberry Pi Pico 2W y dos módulos de comunicación inalámbrica NRF24L01. Cada módulo se conectó a una de las tarjetas mediante el protocolo SPI.

Se realizaron las conexiones correspondientes entre los pines de los módulos y las tarjetas, utilizando las siguientes señales:

- VCC: alimentación de 3,3 V.
- GND: conexión a tierra.
- CE: GPIO 17.
- CSN: GPIO 20.
- SCK: GPIO 18.
- MOSI: GPIO 19.
- MISO: GPIO 16.

Adicionalmente, se conectó una pantalla OLED SSD1306 a cada Raspberry Pi mediante el protocolo I2C. Para esta conexión se utilizaron los pines GP4 como SDA y GP5 como SCL, junto con la alimentación y tierra correspondientes.

También se conservaron los botones físicos conectados a los pines GP10 y GP11, así como el LED externo conectado al pin GP12 mediante una resistencia limitadora de corriente.

2.2. Programación y configuración inicial del NRF24L01:

Una vez realizadas las conexiones, se desarrolló el programa en MicroPython para controlar los módulos NRF24L01 desde las Raspberry Pi.

Se configuró la comunicación SPI con una velocidad de reloj de 1 MHz y se implementaron funciones para escribir y leer los registros internos del módulo. Estas funciones permitieron modificar los parámetros de funcionamiento y ejecutar los comandos necesarios para controlar el dispositivo.

Posteriormente, se configuraron los registros relacionados con la dirección de comunicación, el canal de radiofrecuencia, la velocidad de transmisión, la potencia de salida y el modo de operación.

Para la selección de parámetros se programaron diferentes opciones:

- Canal de operación entre 70 y 80.
- Velocidades de 250 kbps, 1 Mbps y 2 Mbps.
- Potencias de -18 dBm, -12 dBm, -6 dBm y 0 dBm.
- Mensajes de texto predefinidos.
- Modos de transmisión (TX) y recepción (RX).
- Tipos de envío único (UNICO) y continuo (LOOP).

También se estableció una dirección de comunicación de cinco bytes y se configuró el tamaño del contenido transmitido para trabajar con cargas útiles de 32 bytes.

2.3. Configuración mediante botones físicos y visualización en OLED:

En la implementación inicial se utilizaron dos botones físicos para modificar los parámetros del módulo NRF24L01.

El primer botón permitía seleccionar el parámetro que se deseaba modificar, mientras que el segundo permitía cambiar su valor. El programa se encargaba de actualizar la configuración del módulo y mostrar la información correspondiente en la pantalla OLED.

La pantalla se programó para visualizar el canal, la velocidad, la potencia, el modo de operación y el tipo de envío. También se incluyeron funciones para mostrar individualmente cada parámetro durante su selección.

Adicionalmente, se incorporó el control del LED para indicar las actividades asociadas con la transmisión y recepción.

2.4. Implementación de la transmisión y recepción de mensajes:

Se programaron las funciones necesarias para realizar el intercambio de información entre los dos módulos NRF24L01.

Para la transmisión se implementó una función encargada de convertir los mensajes de texto a bytes, ajustar su longitud al tamaño establecido y cargar la información en el registro de transmisión del módulo. Después, se generó el pulso correspondiente en el pin CE para iniciar el envío.

Para la recepción se configuró el segundo módulo en modo RX, habilitando la escucha de paquetes a través del canal seleccionado. Se incorporó una función para consultar el estado del módulo, verificar la disponibilidad de datos y leer la información recibida.

Los mensajes recuperados se decodificaron para convertirlos nuevamente en texto y se enviaron a la consola de Thonny para facilitar su visualización.

2.5. Integración del control del NRF24L01 con una interfaz web:

Una vez establecida la estructura de configuración del NRF24L01, se modificó el sistema para permitir que los parámetros pudieran seleccionarse desde una página web, en lugar de utilizar los botones físicos como mecanismo principal de configuración.

Para esto, se tomó como base el servidor web desarrollado durante la primera parte del laboratorio y se adaptó el programa de MicroPython para integrar las funciones de conectividad WiFi con el control del módulo NRF24L01.

También se modificó el archivo *index.html* para incorporar controles que permitieran seleccionar el canal, la velocidad de transmisión, la potencia, el mensaje, el modo de operación y el tipo de envío. Mediante JavaScript se implementaron las solicitudes HTTP necesarias para enviar las configuraciones seleccionadas al servidor de la Raspberry Pi.

En el programa principal se incorporaron las funciones encargadas de recibir estas solicitudes, actualizar las variables de configuración y aplicar los cambios correspondientes en los registros del NRF24L01.

Las pantallas OLED se conservaron para mostrar los parámetros seleccionados y el estado de funcionamiento de cada dispositivo, mientras que los botones físicos dejaron de utilizarse para modificar la configuración desde el sistema principal.

2.6. Configuración de las redes WiFi independientes:

Para la integración final se configuró cada Raspberry Pi para conectarse a una red WiFi diferente. En el código de cada tarjeta se modificaron los parámetros SSID y PASSWORD, utilizando el nombre y la contraseña correspondientes a la red asignada.

Cada Raspberry Pi ejecutó su propio servidor web y permitió acceder a su interfaz de configuración mediante la dirección IP asignada por su respectiva red inalámbrica.
De esta manera, la conexión WiFi se utilizó para acceder a las páginas de control, mientras que los módulos NRF24L01 mantuvieron un enlace de comunicación por radiofrecuencia independiente entre las dos tarjetas.

2.7. Prueba de comunicación entre los dos módulos:

Finalmente, se realizó la configuración de ambos módulos para establecer la comunicación entre ellos. En la Raspberry Pi transmisora se seleccionó el modo TX, mientras que en la receptora se configuró el modo RX.
Se establecieron parámetros compatibles en ambos dispositivos, incluyendo el canal, la velocidad de transmisión y la dirección de comunicación. Posteriormente, desde la página web de la Raspberry transmisora se seleccionó el mensaje que se deseaba enviar y el tipo de transmisión.
En la Raspberry receptora se habilitó la recepción de paquetes y se utilizó la consola de Thonny para visualizar los mensajes decodificados. También se consultó la información de recepción desde la interfaz web correspondiente.
Durante esta etapa se realizaron cambios en los mensajes y en los parámetros de configuración desde las páginas web, con el propósito de comprobar el funcionamiento del sistema integrado y la interacción entre las interfaces de control, las tarjetas Raspberry Pi y los módulos NRF24L01.

V. ANÁLISIS DE RESULTADOS

1. Primera parte: Implementación y pruebas de conectividad WiFi con Raspberry Pi Pico W

1.1. Identificación de la dirección MAC:

Para comenzar, se realizó la identificación de la dirección MAC de la Raspberry Pi Pico W mediante MicroPython. Para ello, se utilizó la librería *network*, que permite acceder a la interfaz de conexión inalámbrica del dispositivo. Primero, se activó la interfaz Wi-Fi en modo estación (STA) y luego se consultó la dirección MAC mediante la función *wifi.config('mac')*.
El pseudocódigo utilizado fue el siguiente:

```
INICIO
   Importar librería network

   Crear interfaz Wi-Fi en modo estación
   Activar interfaz Wi-Fi

   Consultar dirección MAC del dispositivo

   Convertir dirección MAC a formato hexadecimal
   Mostrar dirección MAC en consola
FIN
```

En la Figura 1 se observa el resultado de la ejecución del programa, donde se obtuvo la dirección MAC *2c:cf:67:dd:71:ab*. Este valor corresponde al identificador físico de la interfaz inalámbrica de la Raspberry Pi Pico 2W y permite reconocer el dispositivo dentro de una red. Se comprobó que la dirección se mantenía igual después de reiniciar la placa.

```
>>> %Run -c $EDITOR_CONTENT

  MPY: soft reboot
  Dirección MAC del Raspberry Pi Pico W: 2c:cf:67:dd:71:ab

>>>
```

Figura 1. Identificación de la dirección MAC de la Raspberry Pi Pico 2W.

1.2. Escaneo de redes WiFi disponibles:

Para esta actividad se utilizó un programa desarrollado en MicroPython, ejecutado desde Thonny en la Raspberry Pi Pico 2W. El programa permitió identificar las redes WiFi disponibles en el entorno mediante la función *wlan.scan()* de la librería *network*, sin necesidad de conectarse a un punto de acceso ni tener acceso a Internet.
Durante el funcionamiento, el programa realizó cinco barridos consecutivos, con una pausa de cinco segundos después de cada escaneo. En cada barrido se obtuvo información de las redes detectadas, como el nombre de la red (SSID), la intensidad de la señal recibida (RSSI), el canal de operación y la dirección MAC del punto de acceso (BSSID). Además, se aplicó un umbral de señal de -80 dBm para filtrar las redes con menor intensidad y se organizaron los resultados de acuerdo con el nivel de RSSI.
El siguiente pseudocódigo resume el funcionamiento del programa:

```
INICIO
   Importar librerías network, time y machine

   Configurar la interfaz WiFi en modo estación
   Activar la interfaz WiFi
   Configurar el umbral mínimo de RSSI (-80 dBm)
   Configurar la pausa entre barridos (5 segundos)

   Inicializar contador de barridos en 0

   REPETIR 5 VECES
      Encender LED indicador

      Iniciar medición del tiempo
      Escanear redes WiFi disponibles
      Finalizar medición del tiempo

      Apagar LED indicador

      Inicializar lista de redes válidas

      PARA cada red detectada
         Obtener SSID, BSSID, canal y RSSI

         Verificar que los datos sean válidos

         SI RSSI es mayor o igual a -80 dBm
            Agregar red a la lista
         FIN SI
      FIN PARA

      Ordenar redes por intensidad de señal
      Mostrar SSID, RSSI, canal y BSSID en consola

      Actualizar información en pantalla OLED
```

Incrementar contador de barridos

Esperar 5 segundos
FIN REPETIR

Desactivar interfaz WiFi
FIN

En las figuras 2, 3, 4, 5 y 6 se muestran los resultados de los 5 barridos:

```
Barrido 1 | inicio t=0.000 s | duracion=10.023 s
SSID                 RSSI (dBm) Canal BSSID
------------------------------------------------------------
UMMG-PUB-CLL100            -38     1 10:f0:68:77:21:21
UMMG-PRIV-CLL100           -40     1 10:f0:68:77:21:20
Redmi Note 14 Pro 5G       -55     1 f2:45:c3:bd:ab:96
UMMG-PUB-CLL100            -67     6 5c:df:89:6a:5d:11
UMMG-PRIV-CLL100           -67     6 5c:df:89:6a:5d:10
UMMG-PUB-CLL100            -70     6 10:f0:68:77:20:c1
UMMG-PRIV-CLL100           -71     6 10:f0:68:77:20:c0
UMMG-PUB-CLL100            -76    11 10:f0:68:77:1d:11
UMMG-PRIV-CLL100           -76    11 10:f0:68:77:1d:10
UMMG-PRIV-CLL100           -80     1 5c:df:89:6a:48:c0
Entradas detectadas: 10 | AP tras filtros: 10 | Invalidas: 0
```

Figura 2. Primer barrido de redes WiFi.

```
Barrido 2 | inicio t=15.045 s | duracion=10.024 s
Tiempo entre inicios: 15.045 s
SSID                 RSSI (dBm) Canal BSSID
------------------------------------------------------------
UMMG-PRIV-CLL100           -41     1 10:f0:68:77:21:20
UMMG-PUB-CLL100            -41     1 10:f0:68:77:21:21
Redmi Note 14 Pro 5G       -57     1 f2:45:c3:bd:ab:96
LABCOM                     -67    11 28:37:37:47:86:36
UMMG-PUB-CLL100            -68     6 10:f0:68:77:20:c1
UMMG-PUB-CLL100            -68     6 5c:df:89:6a:5d:11
UMMG-PRIV-CLL100           -71     6 10:f0:68:77:20:c0
Entradas detectadas: 10 | AP tras filtros: 7 | Invalidas: 0
```

Figura 3. Segundo barrido de redes WiFi.

```
Barrido 3 | inicio t=30.087 s | duracion=10.024 s
Tiempo entre inicios: 15.042 s
SSID                 RSSI (dBm) Canal BSSID
------------------------------------------------------------
UMMG-PRIV-CLL100           -38     1 10:f0:68:77:21:20
UMMG-PUB-CLL100            -42     1 10:f0:68:77:21:21
Redmi Note 14 Pro 5G       -56     1 f2:45:c3:bd:ab:96
PicoW-AP-LS                -57     6 2c:cf:67:dd:7c:b0
UMMG-PUB-CLL100            -65     6 10:f0:68:77:20:c1
UMMG-PUB-CLL100            -65     6 5c:df:89:6a:5d:11
UMMG-PRIV-CLL100           -68     6 10:f0:68:77:20:c0
LABCOM                     -68    11 28:37:37:47:86:36
UMMG-PRIV-CLL100           -76     6 10:f0:68:77:1e:80
UMMG-PRIV-CLL100           -78    11 10:f0:68:77:20:60
UMMG-PUB-CLL100            -78    11 10:f0:68:77:20:61
Entradas detectadas: 13 | AP tras filtros: 11 | Invalidas: 0
```

Figura 4. Tercer barrido de redes WiFi.

```
Barrido 4 | inicio t=45.136 s | duracion=10.024 s
Tiempo entre inicios: 15.049 s
SSID                 RSSI (dBm) Canal BSSID
------------------------------------------------------------
UMMG-PRIV-CLL100           -44     1 10:f0:68:77:21:20
UMMG-PUB-CLL100            -45     1 10:f0:68:77:21:21
PicoW-AP-LS                -61     6 2c:cf:67:dd:7c:b0
UMMG-PRIV-CLL100           -65     6 10:f0:68:77:20:c0
LABCOM                     -66    11 28:37:37:47:86:36
UMMG-PRIV-CLL100           -67     6 5c:df:89:6a:5d:10
UMMG-PUB-CLL100            -69     6 5c:df:89:6a:5d:11
UMMG-PUB-CLL100            -70     6 10:f0:68:77:20:c1
UMMG-PRIV-CLL100           -78     6 10:f0:68:77:1e:80
Entradas detectadas: 9 | AP tras filtros: 9 | Invalidas: 0
```

Figura 5. Cuarto barrido de redes WiFi.

```
Barrido 5 | inicio t=60.179 s | duracion=10.025 s
Tiempo entre inicios: 15.043 s
SSID                 RSSI (dBm) Canal BSSID
------------------------------------------------------------
UMMG-PUB-CLL100            -44     1 10:f0:68:77:21:21
UMMG-PRIV-CLL100           -44     1 10:f0:68:77:21:20
Redmi Note 14 Pro 5G       -55     1 f2:45:c3:bd:ab:96
PicoW-AP-LS                -62     6 2c:cf:67:dd:7c:b0
LABCOM                     -64    11 28:37:37:47:86:36
UMMG-PUB-CLL100            -66     6 5c:df:89:6a:5d:11
UMMG-PRIV-CLL100           -66     6 5c:df:89:6a:5d:10
DIRECT-hE-BRAVIA           -71     7 d2:14:3d:5a:44:6c
UMMG-PUB-CLL100            -78    11 5c:df:89:6a:48:c1
Entradas detectadas: 12 | AP tras filtros: 9 | Invalidas: 0
```

Figura 6. Quinto barrido de redes WiFi.

En los cinco barridos se observó que la cantidad de redes detectadas y sus niveles de RSSI variaron entre una medición y otra. Esto se debe a que el escaneo se realizó en un entorno donde las señales inalámbricas pueden presentar fluctuaciones por interferencias, obstáculos y cambios en las condiciones de propagación. También se identificaron redes que operaban en los canales 1, 6 y 11, correspondientes a la banda de 2,4 GHz. Es importante destacar que el programa no estableció una conexión con las redes encontradas; únicamente realizó un proceso de exploración para obtener información de los puntos de acceso cercanos.

1.2.1. Registro de los puntos de acceso detectados:

A partir de los cinco barridos realizados, se identificaron diferentes puntos de acceso (AP). Para registrar la información solicitada en la guía, se organizaron los cinco AP con mayor intensidad de señal (RSSI) de cada barrido, incluyendo su SSID, RSSI, canal y BSSID en la tabla 1.

Barrido	SSID	RSSI (dBm)	Canal	BSSID
1	UMGG-PUB-CLL100	-38	1	10:f0:68:77:21:21
1	UMGG-PRIV-CLL100	-40	1	10:f0:68:77:21:20
1	Redmi Note 14 Pro 5G	-55	1	f2:45:c3:bd:ab:96
1	UMGG-PUB-CLL100	-67	6	5c:df:89:6a:5d:11
1	UMGG-PRIV-CLL100	-67	6	5c:df:89:6a:5d:10
2	UMGG-PRIV-CLL100	-41	1	10:f0:68:77:21:20
2	UMGG-PUB-CLL100	-41	1	10:f0:68:77:21:21
2	Redmi Note 14 Pro 5G	-57	1	f2:45:c3:bd:ab:96
2	LABCOM	-67	11	28:37:37:47:86:36
2	UMGG-PUB-CLL100	-68	6	10:f0:68:77:20:c1
3	UMGG-PRIV-CLL100	-38	1	10:f0:68:77:21:20
3	UMGG-PUB-CLL100	-42	1	10:f0:68:77:21:21
3	Redmi Note 14 Pro 5G	-56	1	f2:45:c3:bd:ab:96
3	PicoW-AP-LS	-57	6	2c:cf:67:dd:7c:b0
3	UMGG-PUB-CLL100	-65	6	10:f0:68:77:20:c1
4	UMGG-PRIV-CLL100	-44	1	10:f0:68:77:21:20
4	UMGG-PUB-CLL100	-45	1	10:f0:68:77:21:21
4	PicoW-AP-LS	-61	6	2c:cf:67:dd:7c:b0
4	UMGG-PRIV-CLL100	-65	6	10:f0:68:77:20:c0
4	LABCOM	-66	11	28:37:37:47:86:36

5	UMGG-PUB-CLL100	-44	1	10:f0:68:77:21:21
5	UMGG-PRIV-CLL100	-44	1	10:f0:68:77:21:20
5	Redmi Note 14 Pro 5G	-55	1	f2:45:c3:bd:ab:96
5	PicoW-AP-LS	-62	6	2c:cf:67:dd:7c:b0
5	LABCOM	-64	11	28:37:37:47:86:36

Tabla 1. Cinco AP con mayor RSSI en cada barrido.

1.2.2. Comparación de canales y solapamiento en 2,4 GHz:

Al comparar los cinco AP con mayor RSSI de cada barrido, se observa que las redes detectadas operan principalmente en los canales 1, 6 y 11 de la banda de 2,4 GHz.

- Canal 1: concentra varias de las señales más fuertes, especialmente las correspondientes a UMGG-PUB-CLL100, UMGG-PRIV-CLL100 y Redmi Note 14 Pro 5G.
- Canal 6: también presenta varios puntos de acceso, entre ellos PicoW-AP-LS y diferentes AP de las redes UMGG.
- Canal 11: aparece principalmente en la red LABCOM.

Los canales 1, 6 y 11 se utilizan habitualmente para reducir el solapamiento entre canales en redes WiFi de 2,4 GHz. Sin embargo, cuando varios puntos de acceso operan en un mismo canal, pueden competir por el medio inalámbrico y generar mayor ocupación del canal.

Esto puede ocasionar interferencias, congestión, disminución de la velocidad de transmisión y aumento de los tiempos de espera para acceder al medio. Por lo tanto, aunque se identificaron los tres canales principales, se observa una mayor concentración de puntos de acceso en el canal 1 y el canal 6.

1.2.3. Variación del RSSI entre los barridos:

Al comparar los resultados de los cinco barridos, se evidencia que la intensidad de señal RSSI presentó variaciones, incluso para los puntos de acceso que mantuvieron el mismo BSSID.

Por ejemplo, para el punto de acceso UMGG-PUB-CLL100, identificado con el BSSID *10:f0:68:77:21:21*, se obtuvieron los siguientes valores, como se muestra en la figura 7.

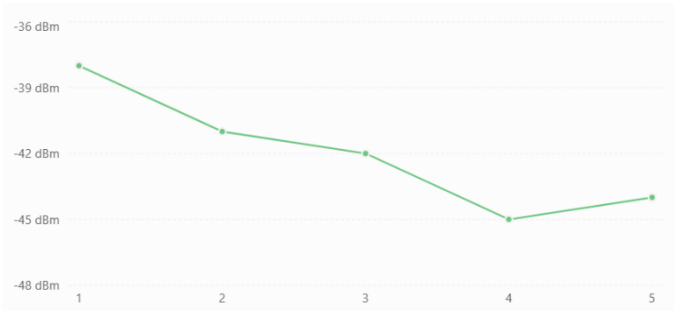


Figura 7. Variación del RSSI entre barridos para el punto de acceso UMGG-PUB-CLL100.

Se observa una disminución progresiva de la intensidad recibida desde -38 dBm hasta -45 dBm, con una pequeña recuperación en el quinto barrido. Aunque las variaciones no son demasiado grandes, demuestran que el nivel de señal recibido no permanece completamente constante.

1.3. Implementación del servidor web y control del LED:

En esta etapa se implementó un servidor web utilizando la Raspberry Pi Pico 2W, con el propósito de controlar el LED integrado de la placa mediante una página web. Para ello, se utilizaron dos archivos: *APWifipico.py*, encargado de configurar el punto de acceso WiFi y gestionar las solicitudes HTTP, e *index.html*, que contiene la interfaz gráfica desde la cual se realizan las acciones de encendido y apagado.

Pseudocódigo del programa APWifipico.py:

```
INICIO
  Importar librerías network, socket, time y gc
  Importar Pin desde machine

  Definir nombre y contraseña de la red WiFi
  Configurar canal y dirección IP
  Definir archivo index.html

  Configurar LED como salida
  Apagar LED

  Crear interfaz WiFi en modo punto de acceso
  Configurar SSID, contraseña y canal
  Asignar dirección IP y máscara de subred
  Activar punto de acceso

  Abrir archivo index.html
  Leer contenido de la página web

  Crear socket del servidor
  Asociar socket con puerto 80
  Poner servidor en escucha

  MIENTRAS el servidor esté activo
    Esperar una solicitud del navegador
    Aceptar conexión

    Leer método HTTP y ruta solicitada

    SI la ruta es "/on"
      Encender LED
      Enviar respuesta con estado ON

    SINO SI la ruta es "/off"
      Apagar LED
      Enviar respuesta con estado OFF

    SINO SI la ruta es "/state"
      Consultar estado actual del LED
      Enviar estado al navegador

    SINO SI la ruta es "/" o "/index.html"
      Enviar archivo HTML al navegador

    SINO
      Enviar respuesta de error
    FIN SI

    Cerrar conexión
    Liberar memoria

  FIN MIENTRAS

  Cerrar servidor
  Apagar LED
```

Desactivar punto de acceso
FIN

Ambos archivos se cargaron directamente en la memoria de la Raspberry Pi Pico 2W mediante Thonny, ubicándolos en la raíz del dispositivo. Luego, se ejecutó el programa en MicroPython para activar el punto de acceso inalámbrico con los parámetros que se muestran en la figura 8.

```
# Comunicaciones Digitales - UMNG
# Pico W / Pico 2 W, MicroPython. Copiar junto con index.html a la raiz.
# Red local: no proporciona acceso a Internet. No requiere Microdot.
import network
import socket
import time
import gc
from machine import Pin

SSID = 'Pico2WEYPGRUPO5'
PASSWORD = '12345678'
CHANNEL = 5
IP = '192.168.4.1'
MASK = '255.255.255.0'
HTML FILE = 'index.html'
```

Figura 8. Configuración del punto de acceso WiFi en el programa de MicroPython.

Una vez iniciado el servidor, se realizó la conexión desde un computador a la red WiFi creada por la Raspberry Pi Pico W. Como se observa en la figura 9, la red *Pico2WEYPGRUPO5* apareció disponible y el computador logró conectarse correctamente. Después, se ingresó desde el navegador a la dirección http://192.168.4.1, donde se visualizó la página web desarrollada para el control del LED.

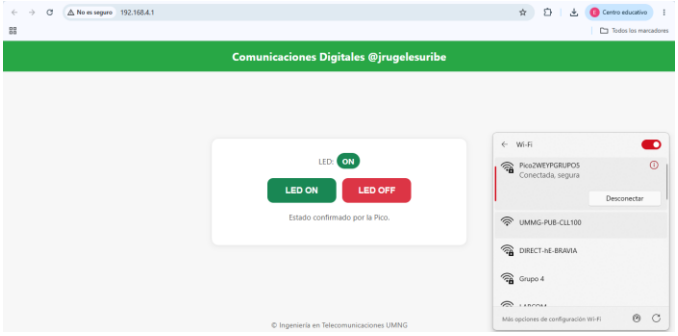


Figura 9. Conexión del computador a la red WiFi creada por la Raspberry Pi Pico 2W.

La interfaz dispone de dos botones: LED ON y LED OFF. Al presionar cualquiera de ellos, la página envía una solicitud HTTP al servidor de la Raspberry Pi, que interpreta la petición y modifica el estado del LED integrado. Además, la página consulta periódicamente el estado del LED para mostrar si se encuentra encendido o apagado.

En las pruebas realizadas se comprobó el funcionamiento del sistema de manera satisfactoria. Primero, se verificó que el computador pudiera conectarse a la red inalámbrica creada por la Raspberry Pi Pico 2W y acceder a la interfaz web mediante la dirección IP configurada. Luego, se utilizaron los botones de la página para encender y apagar el LED integrado de la placa, como se observa en la figura 10. En ambos casos, el cambio de estado se observó físicamente en la Raspberry Pi y fue confirmado en la interfaz del navegador.

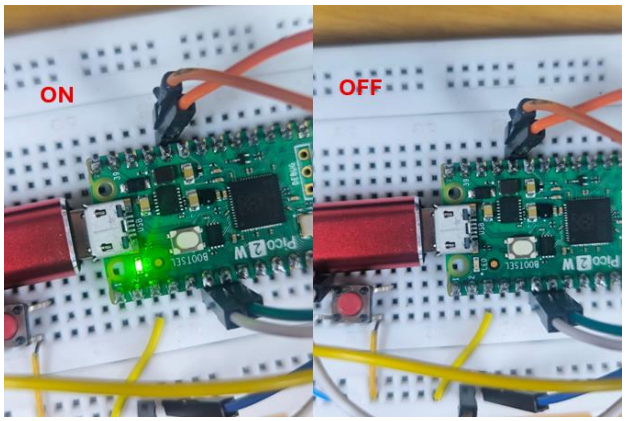


Figura 10. Montaje de la Raspberry Pi Pico W utilizado para la prueba de control del LED.

De esta manera, se comprobó que era posible establecer una comunicación entre el navegador y la Raspberry Pi Pico W mediante una red WiFi local, utilizando solicitudes HTTP para controlar un elemento físico de la placa, sin requerir conexión a Internet.

1.4. Observación de la señal WiFi mediante el analizador de espectro:

En esta etapa se buscó observar el comportamiento de la señal WiFi-generada por la Raspberry Pi Pico 2W mediante un analizador de espectro Anritsu. Para ello, se utilizó un programa en MicroPython y una página web diseñada para generar tráfico inalámbrico desde la Raspberry Pi hacia un computador conectado a su punto de acceso.

A diferencia de la página web utilizada en el punto anterior, esta versión incorporó una sección denominada *«Prueba de tráfico WiFi»*, que permitió seleccionar la duración de las pruebas y ejecutar dos modalidades de transmisión: intermitente y sostenida.

Para realizar el procedimiento, se cargaron en la Raspberry Pi los archivos *TrafficWifi.py* e *index.html*. El primero se encargó de configurar el punto de acceso, iniciar el servidor HTTP y transmitir bloques de datos cuando el computador realizaba solicitudes. El segundo proporcionó la interfaz desde la cual se seleccionaba el tipo y la duración de la prueba.

Pseudocódigo del programa TrafficWifi.py:
INICIO
Importar librerías network, socket, time y gc
Importar Pin desde machine

Definir SSID, contraseña, canal y dirección IP
Configurar tamaño de los bloques de datos

Configurar LED como salida
Apagar LED

Crear punto de acceso WiFi
Asignar SSID, contraseña y canal
Configurar dirección IP
Activar punto de acceso

Cargar archivo index.html

Crear servidor mediante socket
Asociar servidor al puerto 80

Activar escucha de conexiones

MIENTRAS el servidor esté activo
Esperar solicitud del computador
Aceptar conexión
Leer método HTTP y ruta solicitada

SI la ruta es "/traffic"
Preparar respuesta de datos binarios
Enviar bloques sucesivos de información
Finalizar transferencia

SINO SI la ruta es "/on" o "/off"
Cambiar estado del LED
Enviar confirmación

SINO SI la ruta es "/state"
Consultar estado del LED
Enviar respuesta

SINO SI la ruta es "/" o "/index.html"
Enviar página web al navegador
FIN SI

Cerrar conexión
Liberar memoria
FIN MIENTRAS

Cerrar servidor
Apagar LED
Desactivar punto de acceso
FIN

Las pruebas se realizaron con duraciones de 30 y 60 segundos, utilizando las siguientes modalidades:

- Transmisión intermitente: enviaba datos durante intervalos de 2 segundos, seguidos de pausas de 3 segundos.
- Transmisión sostenida: realizaba descargas sucesivas de datos sin pausas programadas.

La interfaz también permitió visualizar el tiempo transcurrido, la cantidad de información recibida y la velocidad promedio de transferencia, como se observa en la figura 11. De esta manera, se generó actividad inalámbrica durante los intervalos seleccionados, facilitando su observación en el analizador de espectro. Estas funciones se encuentran implementadas en el código de la página web y en el servidor MicroPython.

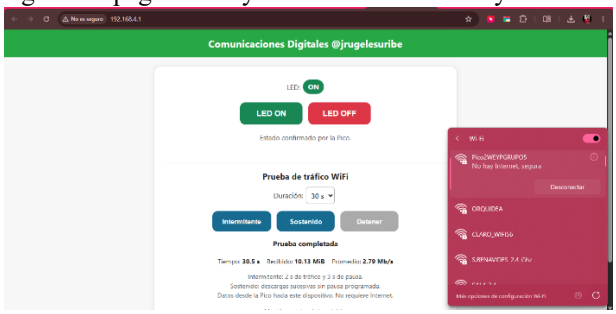


Figura 11. Interfaz web para la generación de tráfico WiFi.

Para observar las señales generadas durante las pruebas, se configuró el analizador de espectro alrededor de la frecuencia de operación del canal WiFi utilizado, canal 5, cuya frecuencia central es de 2432 MHz.

Se ajustó el rango de frecuencias para observar la actividad alrededor de esta frecuencia y se utilizó la función de espectrograma del analizador. Esta representación permitió observar cómo variaba la actividad de la señal a lo largo del tiempo durante las transmisiones intermitentes y sostenidas, como se muestra en la figura 12.

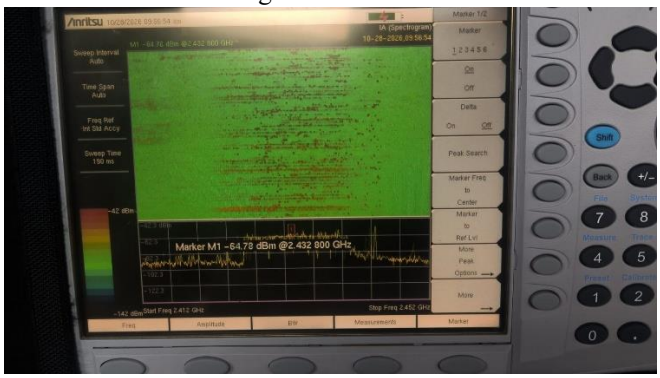


Figura 12. Captura del espectrograma de la señal WiFi mediante el analizador de espectro.

En la figura 12 se observa el espectrograma de la señal WiFi capturada durante las pruebas de transmisión. En esta representación, las líneas y marcas de color rojizo corresponden a los eventos de transmisión de datos que fueron detectados por el analizador alrededor de la frecuencia de operación. Estas aparecen distribuidas a lo largo del espectrograma y permiten identificar los momentos en los que se presentó actividad inalámbrica.

La intensidad y concentración de estas marcas varían con el tiempo, debido a que la transmisión de información no mantiene una potencia constante. Durante los intervalos en los que se enviaron datos, se observa una mayor presencia de líneas rojizas, mientras que los cambios en su distribución reflejan las variaciones de actividad de la señal. Esto permite distinguir visualmente las transmisiones y relacionarlas con las pruebas de tráfico intermitente y sostenido realizadas desde la interfaz web.

Adicionalmente, el marcador registró un nivel de -64,78 dBm a 2432,8 MHz, frecuencia cercana a la frecuencia central del canal 5 (2432 MHz). Por lo tanto, la captura permitió comprobar que el analizador detectó actividad de radiofrecuencia en la región donde se estaba generando el tráfico WiFi.

1.5. Lectura del convertidor analógico-digital (ADC):

En esta etapa se realizó la lectura del convertidor analógico-digital (ADC) de la Raspberry Pi Pico 2W, utilizando el canal ADC0, correspondiente al pin GP26. El objetivo fue obtener una lectura digital de una señal analógica y visualizar tanto el valor obtenido como su tensión estimada mediante una página web.

Para desarrollar esta actividad se utilizaron dos archivos: *APWifipico5.py*, encargado de configurar la Raspberry Pi y realizar la lectura del ADC, e *index5.html*, que contiene la interfaz web para mostrar los datos recibidos. Ambos archivos se cargaron en la memoria de la Raspberry Pi Pico W mediante Thonny.

En el programa de MicroPython se importó la clase ADC desde la librería machine y se configuró el pin GP26 como entrada analógica mediante la instrucción ADC(26). Luego, se habilitó

el punto de acceso WiFi y el servidor web, permitiendo que un computador conectado a la red pudiera consultar las mediciones a través del navegador.
La lectura del ADC se realizó mediante la función *read_u16()*, que entrega un valor digital de 16 bits. A partir de este resultado, el programa calculó una tensión estimada utilizando una referencia de 3,3 V y envió ambos valores en formato JSON hacia la página web. La interfaz se encargó de actualizar automáticamente la información cada segundo.

Pseudocódigo del programa *APWifipico5.py*:

```
INICIO
  Importar librerías network, socket, time y gc
  Importar Pin y ADC desde machine

  Definir parámetros de la red WiFi
  Configurar dirección IP y servidor HTTP

  Configurar LED como salida
  Configurar ADC0 en el pin GP26

  Crear punto de acceso WiFi
  Activar red inalámbrica

  Cargar archivo index5.html

  Crear servidor web en el puerto 80

  MIENTRAS el servidor esté activo
    Esperar solicitud del navegador
    Aceptar conexión
    Leer solicitud HTTP

    SI la ruta solicitada es "/adc"
      Leer valor digital del ADC0
      Calcular tensión estimada:
        Voltaje = Valor digital * 3.3 / 65535

      Crear respuesta JSON
      Enviar valor digital y voltaje al navegador

    SINO SI la ruta es "/on" o "/off"
      Controlar estado del LED
      Enviar confirmación

    SINO SI la ruta es "/state"
      Consultar estado del LED
      Enviar respuesta

    SINO SI la ruta es "/" o "/index.html"
      Enviar página web
    FIN SI

    Cerrar conexión
    Liberar memoria
  FIN MIENTRAS

  Cerrar servidor
  Apagar LED
  Desactivar punto de acceso
FIN
```

En la figura 13 se observa la página web después de establecer la conexión con la Raspberry Pi Pico W. En la sección denominada «Lectura del ADC0 (GP26)» se muestran los valores recibidos desde la placa, correspondientes a la conversión analógico-digital y a la tensión estimada.



Figura 13. Interfaz web para la visualización de las lecturas del ADC0.

Durante la prueba se obtuvo un valor digital de 12563, correspondiente a una tensión estimada de 0,633 V en la entrada analógica GP26.
La relación utilizada para calcular la tensión fue:

$$V_{in} = \frac{ADC}{65535} * 3.3V$$
$$V_{in} = \frac{12563}{65535} * 3.3V = 0.633V$$

El resultado demuestra que la Raspberry Pi Pico 2W realizó correctamente la conversión de la señal analógica a un valor digital y permitió obtener una estimación de la tensión presente en la entrada ADC0.
Adicionalmente, se comprobó que los datos podían transmitirse desde la Raspberry Pi hasta el computador mediante la red WiFi local. La página web mostró tanto el valor digital como la tensión estimada y confirmó que la lectura se estaba recibiendo correctamente desde la placa.
La actualización automática cada segundo permitió visualizar las mediciones sin necesidad de ejecutar manualmente el programa para cada lectura. De esta manera, se integraron las funciones de adquisición analógica, procesamiento de datos y visualización mediante una interfaz web.

1.6. Medición de RSSI en función de la distancia:

En esta etapa se evaluó el comportamiento de la intensidad de la señal WiFi (RSSI) respecto a la distancia entre la Raspberry Pi Pico 2W y el punto de acceso inalámbrico. El objetivo fue observar cómo variaba la potencia recibida al aumentar la separación entre los dispositivos y determinar el alcance máximo de comunicación alcanzado durante la práctica.
Inicialmente, se realizó un barrido de las redes WiFi disponibles para identificar el punto de acceso que se utilizaría en las mediciones, como se muestra en la figura 14. Se seleccionó la red EDIEMYPAULA, correspondiente al punto de acceso del celular, y se trabajó con ella como red objetivo.

```
Barrido 7 | inicio t=34.406 s | duracion=0.729 s
Tiempo entre inicios: 5.739 s
SSID                 RSSI (dBm) Canal BSSID
------------------------------------------------------------
EDIEMYPAULA                -30     1 7e:19:fa:20:49:5c
UMMG-PRIV-CLL100           -59    11 94:b3:4f:5b:3b:b0
UMMG-PUB-CLL100            -60    11 94:b3:4f:5b:3b:b1
HONOR X5c Plus             -77     6 22:d8:a9:94:3f:70
UMMG-PUB-CLL100            -80     1 10:f0:68:77:20:d1
Entradas detectadas: 5 | AP tras filtros: 5 | Invalidas: 0
```

Figura 14. Barrido de las redes Wifi disponibles.

Luego, se ejecutó el programa *Punto 3.5.py* en MicroPython desde Thonny. Este programa permitió buscar la red seleccionada mediante su SSID, obtener el valor RSSI y realizar diez mediciones en cada distancia. Después, calculó el promedio de las muestras válidas y almacenó los resultados en un archivo CSV llamado *rssi_distancia5.csv.*

Pseudocódigo del programa *Punto 3.5.py*:

INICIO
Importar librerías network, time y math

Definir SSID objetivo: EDIEMYPAULA
Definir número de muestras: 10
Definir tiempo de espera: 1 segundo

Activar interfaz WiFi en modo estación

Realizar barrido de redes WiFi disponibles
Identificar la red EDIEMYPAULA

Crear archivo CSV
Escribir encabezados de las mediciones

REPETIR
Solicitar distancia en metros

SI distancia = 0
Finalizar medición
FIN SI

Inicializar lista de muestras

REPETIR 10 VECES
Escanear redes WiFi disponibles
Buscar la red EDIEMYPAULA

SI la red es detectada
Obtener valor RSSI
Guardar muestra
SINO
Registrar que no fue detectada
FIN SI

Esperar 1 segundo
FIN REPETIR

Calcular promedio de las muestras válidas

Guardar distancia, muestras y promedio en CSV

Mostrar resultados en consola

HASTA que se indique finalizar

Cerrar archivo CSV
Mostrar mensaje de prueba finalizada
FIN

Las primeras mediciones se realizaron a distancias de 1, 2, 3, 4 y 5 metros, como se muestra en la figura 15, 16, 17, 18 y 19. Luego se guardaron los datos en un archivo csv.

```
Midiendo a 1.0 metros
Muestra 1 : -40 dBm
Muestra 2 : -40 dBm
Muestra 3 : -43 dBm
Muestra 4 : -40 dBm
Muestra 5 : -43 dBm
Muestra 6 : -40 dBm
Muestra 7 : -43 dBm
Muestra 8 : -40 dBm
Muestra 9 : -40 dBm
Muestra 10 : -40 dBm
Muestras validas: 10
RSSI promedio: -40.9 dBm
Datos guardados en rssi_distancia5.csv
```

Figura 15. Medida realizada a 1m de distancia.

```
Midiendo a 2.0 metros
Muestra 1 : -42 dBm
Muestra 2 : -50 dBm
Muestra 3 : -52 dBm
Muestra 4 : -50 dBm
Muestra 5 : -48 dBm
Muestra 6 : -50 dBm
Muestra 7 : -50 dBm
Muestra 8 : -51 dBm
Muestra 9 : -50 dBm
Muestra 10 : -50 dBm
Muestras validas: 10
RSSI promedio: -49.3 dBm
Datos guardados en rssi_distancia5.csv
```

Figura 16. Medida realizada a 2m de distancia.

```
Midiendo a 3.0 metros
Muestra 1 : -52 dBm
Muestra 2 : -60 dBm
Muestra 3 : -59 dBm
Muestra 4 : -59 dBm
Muestra 5 : -59 dBm
Muestra 6 : -61 dBm
Muestra 7 : -60 dBm
Muestra 8 : -60 dBm
Muestra 9 : -66 dBm
Muestra 10 : -61 dBm
Muestras validas: 10
RSSI promedio: -59.7 dBm
Datos guardados en rssi_distancia5.csv
```

Figura 17. Medida realizada a 3m de distancia.

```
Midiendo a 4.0 metros
Muestra 1 : -56 dBm
Muestra 2 : -68 dBm
Muestra 3 : -70 dBm
Muestra 4 : -65 dBm
Muestra 5 : -68 dBm
Muestra 6 : -70 dBm
Muestra 7 : -73 dBm
Muestra 8 : -65 dBm
Muestra 9 : -67 dBm
Muestra 10 : -60 dBm
Muestras validas: 10
RSSI promedio: -66.2 dBm
Datos guardados en rssi_distancia5.csv
```

Figura 18. Medida realizada a 4m de distancia.

```
Midiendo a 5.0 metros
Muestra 1 : -59 dBm
Muestra 2 : -61 dBm
Muestra 3 : -64 dBm
Muestra 4 : -62 dBm
Muestra 5 : -64 dBm
Muestra 6 : -67 dBm
Muestra 7 : -62 dBm
Muestra 8 : -65 dBm
Muestra 9 : -62 dBm
Muestra 10 : -62 dBm
Muestras validas: 10
RSSI promedio: -62.8 dBm
Datos guardados en rssi_distancia5.csv
```

Figura 19. Medida realizada a 5m de distancia.

Los resultados obtenidos durante la primera etapa se resumen en la tabla 2:

Distancia (m)	Muestras válidas	RSSI promedio (dBm)
1	10	-40,9
2	10	-49,3
3	10	-59,7
4	10	-66,2
5	10	-62,8

Tabla 2. Valores promedio de RSSI para diferentes distancias.

En la figura 20 se muestra la gráfica obtenida del comportamiento del RSSI respecto a la distancia.

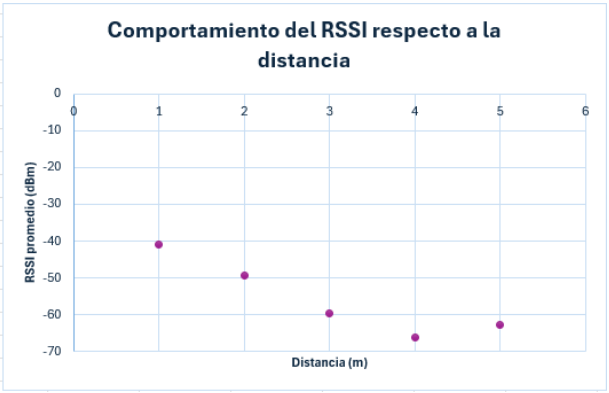


Figura 20. Gráfica del comportamiento del RSSI respecto a la distancia.

Al analizar las mediciones realizadas entre 1 y 5 metros, se observa una tendencia general de disminución del RSSI a medida que aumenta la distancia entre el punto de acceso y la Raspberry Pi Pico 2W. A un metro se obtuvo un promedio de -40,9 dBm, mientras que a cuatro metros el valor disminuyó hasta -66,2 dBm. Esto demuestra que la potencia de la señal recibida se reduce conforme aumenta la separación entre los dispositivos.
Sin embargo, a los cinco metros se registró un RSSI promedio de -62,8 dBm, ligeramente superior al obtenido a los cuatro metros. Esta variación no significa que la señal mejore necesariamente con la distancia, sino que puede estar relacionada con los obstáculos presentes en el entorno y las variaciones propias de las mediciones inalámbricas.

Después de realizar las mediciones iniciales, se aumentó progresivamente la distancia entre los dispositivos hasta alcanzar los 66 metros. En este punto se obtuvieron cuatro muestras válidas, como se muestra en la tabla 3.

Muestra	RSSI (dBm)
1	-85
2	-84
3	-94
4	-85
Promedio	-87

Tabla 3. Valores para el alcance máximo.

A esta distancia, la Raspberry Pi todavía logró detectar la red EDIEMYPAULA, aunque con una intensidad de señal considerablemente menor que la registrada en las primeras mediciones.
La Raspberry logró detectar la red a una distancia de 66 metros, con un RSSI promedio de -87 dBm. Este resultado permitió establecer que el alcance máximo alcanzado experimentalmente durante la práctica fue de 66 metros.
A esta distancia, la intensidad de la señal era considerablemente menor que en los puntos iniciales, pero todavía se podían obtener lecturas del punto de acceso. Es importante aclarar que este valor corresponde al máximo alcanzado durante las pruebas y no necesariamente al límite físico de alcance del dispositivo.

A partir de los promedios registrados entre 1 y 5 metros, se realizó un ajuste lineal para representar matemáticamente el comportamiento del RSSI en función de la distancia.

La ecuación obtenida fue:

$$RSSI(d) = -6.07d - 37.57$$

El coeficiente de determinación obtenido fue:

$$R^2 = 0.844$$

Este valor indica que el ajuste lineal representa razonablemente la tendencia de los datos medidos entre 1 y 5 metros. Sin embargo, debido a las variaciones propias de la propagación inalámbrica, la ecuación debe entenderse como una aproximación basada en las mediciones realizadas y no como un modelo exacto para cualquier distancia.

2. Segunda parte: Implementación de pruebas de comunicación con módulos NRF24L01

Para esta etapa del laboratorio se implementó un sistema de comunicación inalámbrica utilizando dos módulos NRF24L01 conectados a dos placas Raspberry Pi Pico W. El propósito fue establecer una comunicación entre ambas placas, permitiendo transmitir y recibir mensajes mediante radiofrecuencia.
Para controlar los módulos se utilizaron dos archivos principales: PicoNRF_Web.py, desarrollado en MicroPython, e index_nrf.html, que contiene la interfaz web para configurar los parámetros de comunicación. Ambos archivos se cargaron en las dos Raspberry Pi, modificando en cada una las credenciales de la red WiFi a la que debía conectarse.
El programa en MicroPython se encargó de inicializar el módulo NRF24L01 mediante el protocolo SPI, configurar la pantalla OLED SSD1306 y establecer la conexión WiFi. Posteriormente, inició un servidor web que permitió acceder a

la interfaz desde un navegador y modificar los parámetros del módulo de radio.
La página web permitió seleccionar el canal RF, la velocidad de transmisión, la potencia, el modo de funcionamiento (TX o RX), el mensaje que se deseaba transmitir y el tipo de envío (único o continuo).

Pseudocódigo del programa *PicoNRF_Web.py*:

```
INICIO

  Importar librerías:
    network, socket, time, gc y json
    Pin, SPI e I2C
    Librería SSD1306

  Definir credenciales WiFi
  Definir puerto del servidor web

  Configurar pines SPI del NRF24L01
  Inicializar comunicación SPI

  Configurar pantalla OLED mediante I2C
  Configurar LED indicador

  Definir registros y comandos del NRF24L01

  Establecer parámetros iniciales:
    Canal RF
    Velocidad de datos
    Potencia de transmisión
    Mensaje
    Modo TX o RX
    Tipo de envío

  FUNCIÓN configure_radio()
    Desactivar módulo NRF
    Configurar registros de radio
    Desactivar Auto-ACK
    Configurar dirección de comunicación
    Configurar canal, velocidad y potencia

    SI modo es TX
      Configurar módulo como transmisor
    SINO
      Configurar módulo como receptor
      Activar recepción
    FIN SI

    Actualizar pantalla OLED
  FIN FUNCIÓN

  FUNCIÓN transmit(mensaje)
    Verificar que el modo sea TX
    Convertir mensaje a bytes
    Ajustar carga útil a 32 bytes
    Limpiar buffer de transmisión
    Cargar mensaje en NRF24L01
    Activar transmisión
    Registrar evento de envío
  FIN FUNCIÓN

  FUNCIÓN receive()
    Verificar que el modo sea RX
    Consultar registro de estado

    SI existe un paquete recibido
      Leer los datos del módulo
      Convertir datos a texto
      Guardar último mensaje recibido
      Mostrar mensaje en consola
      Encender LED indicador
      Apagar LED indicador
    FIN SI
  FIN FUNCIÓN

  FUNCIÓN wifi_connect()
    Activar interfaz WiFi
    Conectarse a la red configurada
    Obtener dirección IP
    Mostrar dirección IP en consola
  FIN FUNCIÓN

  FUNCIÓN handle_client()
    Recibir solicitud HTTP

    SI solicitan la página principal
      Enviar archivo index.html

    SINO SI solicitan estado
      Enviar configuración y último mensaje

    SINO SI solicitan cambiar configuración
      Leer parámetros enviados desde la página
      Validar los valores
      Aplicar nueva configuración
      Actualizar módulo NRF y pantalla OLED

    SINO SI solicitan transmitir
      Verificar modo TX
      Enviar mensaje
    FIN SI

    Cerrar conexión
  FIN FUNCIÓN

  Inicializar pantalla OLED
  Configurar módulo NRF24L01
  Conectarse a la red WiFi

  Crear servidor web
  Escuchar solicitudes HTTP

  MIENTRAS el programa esté activo

    Atender solicitudes del navegador
    Revisar si existen mensajes recibidos

    SI modo es TX y tipo de envío es LOOP
      Transmitir mensaje cada 250 ms

    SINO SI modo es TX y tipo de envío es UNICO
      Transmitir una sola vez
      cuando se aplica una nueva configuración
    FIN SI

    Esperar breve intervalo

  FIN MIENTRAS

FIN
```

Para comprobar el funcionamiento del sistema se configuraron las dos Raspberry Pi con parámetros complementarios: una placa trabajó en modo transmisor (TX) y la otra en modo receptor (RX). Cada una se conectó a una red WiFi diferente y obtuvo su propia dirección IP, lo que permitió acceder de manera independiente a la interfaz web alojada en cada dispositivo.

En la figura 21 se observa la configuración de la Raspberry Pi encargada de transmitir. En el código de MicroPython se encuentran definidas las credenciales de la red WiFi y las conexiones del módulo NRF24L01.

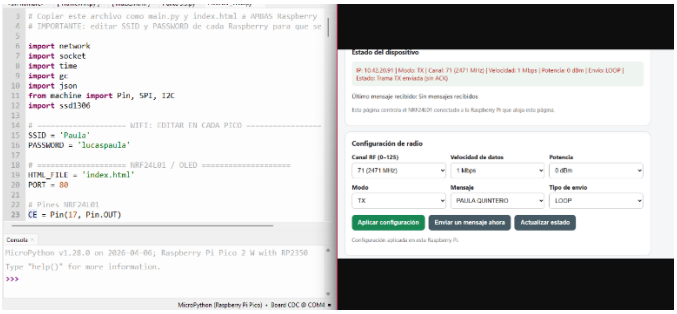


Figura 21. Configuración del módulo NRF24L01 en modo transmisor (TX).

La interfaz permitió aplicar la configuración y ejecutar el envío del mensaje de manera continua. En el estado del dispositivo se visualizó el evento *«Trama TX enviada (sin ACK)»*, indicando que el módulo había ejecutado la transmisión.

En la figura 22 se observa la configuración de la Raspberry Pi que trabajó como receptora. Esta placa se conectó a una red WiFi diferente y se configuró con los mismos parámetros de radio utilizados en el transmisor: canal 71 (2471 MHz), velocidad de 1 Mbps y potencia de 0 dBm. La diferencia principal fue la selección del modo RX.

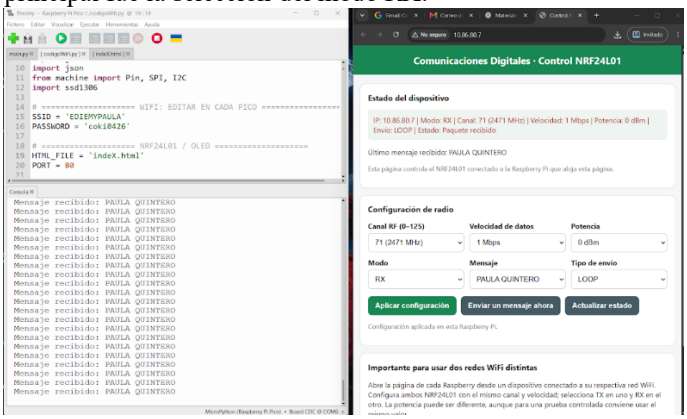


Figura 22. Configuración del módulo NRF24L01 en modo receptor (RX) y recepción del mensaje.

En la consola de Thonny se observan repetidamente los mensajes Mensaje recibido: PAULA QUINTERO, lo que demuestra que el módulo receptor detectó y procesó correctamente los paquetes enviados por el transmisor. Además, en la interfaz web aparece el mensaje *«Último mensaje recibido: PAULA QUINTERO»* y el estado *«Paquete recibido»*.

Este resultado constituye la evidencia principal de que se estableció correctamente la comunicación entre los dos módulos NRF24L01.

Al comparar las dos configuraciones se evidencia que ambos módulos trabajaron con parámetros de radio compatibles, especialmente en el canal, la velocidad de transmisión y la dirección de comunicación. Esto permitió que el receptor pudiera interpretar los paquetes enviados por el transmisor.
También se comprobó que las redes WiFi utilizadas por las dos Raspberry Pi no necesitaban ser la misma red. La conexión WiFi se utilizó para acceder a las páginas de configuración de cada placa, mientras que el intercambio de mensajes se realizó mediante los módulos NRF24L01. Por lo tanto, ambas comunicaciones funcionaron de manera independiente.
Otro aspecto importante fue la integración de la interfaz web con el programa de MicroPython. Esta permitió modificar los parámetros de radio sin depender de los botones físicos, mientras que la pantalla OLED se mantuvo como medio de visualización de la configuración del módulo.
Finalmente, la recepción repetida del mensaje PAULA QUINTERO permitió verificar el funcionamiento del modo LOOP, en el cual el transmisor envía el mensaje aproximadamente cada 250 ms. De esta manera, se comprobó la transmisión y recepción de información entre las dos Raspberry Pi mediante los módulos NRF24L01.

VI. CONCLUSIONES

- Se exploraron las funciones WiFi de la Raspberry Pi Pico 2W mediante MicroPython, identificando su dirección MAC y realizando cinco barridos para detectar redes cercanas. Se obtuvo información de los puntos de acceso, como SSID, RSSI, canal y BSSID, observando variaciones en las señales detectadas.
- Se implementó un servidor web que permitió crear un punto de acceso WiFi y controlar el LED integrado de la Raspberry Pi desde un computador. Se comprobó el encendido y apagado mediante una interfaz web, utilizando solicitudes HTTP sin necesidad de conexión a Internet.
- Las pruebas de tráfico WiFi intermitente y sostenido permitieron generar y observar señales mediante el analizador de espectro. En el espectrograma se identificaron las transmisiones mediante marcas rojizas y se registró un nivel de -64,78 dBm a 2432,8 MHz.
- Se comprobó el funcionamiento del ADC0 (GP26), obteniendo un valor digital de 12563, equivalente a una tensión estimada de 0,633 V. Estos datos se visualizaron y actualizaron automáticamente en la página web, integrando la adquisición y transmisión de información mediante WiFi.
- Las mediciones de RSSI entre 1 y 5 metros evidenciaron una tendencia general de disminución de la intensidad de señal con la distancia. Se obtuvo la ecuación aproximada, y se alcanzó una distancia máxima experimental de 66 metros, con un RSSI promedio de -87 dBm.
- En la segunda parte se implementó un sistema de comunicación con dos módulos NRF24L01 y dos

Raspberry Pi. Mediante una interfaz web fue posible configurar el canal, la velocidad, la potencia, el modo de operación y los mensajes, manteniendo la pantalla OLED para visualizar los parámetros seleccionados.

- Se verificó la transmisión y recepción del mensaje PAULA QUINTERO utilizando el canal 71 (2471 MHz), una velocidad de 1 Mbps y una potencia de 0 dBm. La recepción repetida del mensaje en la consola y en la interfaz web confirmó la comunicación entre los módulos, incluso cuando las Raspberry Pi estaban conectadas a redes WiFi diferentes.

VII. BIBLIÓGRAFIA

[1] Murata Manufacturing, «Conceptos básicos de las comunicaciones inalámbricas (1)», [En línea]. Disponible en: https://article.murata.com/es/article/basics-of-wireless-communication-1. [Último acceso: 1 de noviembre de 2026].

[2] Raspberry Pi Ltd., «Raspberry Pi Pico 2», [En línea]. Disponible en: https://www.raspberrypi.com/products/raspberry-pi-pico-2/. [Último acceso: 1 de noviembre de 2026].

[3] MicroPython, «network.WLAN — Controlador de red», [En línea]. Disponible en: https://docs.micropython.org/en/latest/library/network.WLAN.html.

[4] MicroPython, «machine.ADC — Conversión analógica a digital», [En línea]. Disponible en: https://docs.micropython.org/en/latest/library/machine.ADC.html. [Último acceso: 1 de noviembre de 2026].

[5] R. Carrasco, «RSSI y rendimiento de antenas y aplicaciones inalámbricas de IoT», Data Alliance, [En línea]. Disponible en: https://www.data-alliance.net/es/rssi-y-rendimiento-de-antenas-y-aplicaciones-inal%C3%A1mbricas-de-iot. [Último acceso: 1 de noviembre de 2026].

[6] Nordic Semiconductor, «nRF24L01+ Product Specification», [En línea]. Disponible en: https://devzone.nordicsemi.com/cfs-file/__key/communityserver-discussions-components-files/4/content.pdf. [Último acceso: 1 de noviembre de 2026].

VIII. ANEXOS

Github Paula Quintero:
https://github.com/paulaquintero2811/Tecnolog-a-WiFi-con-Raspberry-Pi-Pico-2W

Github Ediem Valero:
https://github.com/estediemvalero-coder/Tecnolog-a-WiFi-con-Raspberry-Pi-Pico-2W.